

[HID](#) [Rotary Encoder](#) [Example](#) [Code](#) [Pins](#)[Dr. Tobias Weltner](#) | [Feb 27, 2024](#)

Using Mechanical Rotary Encoder

Use a Simple Mechanical Rotary Encoder Directly With A Microprocessor. No Need For Breakout Boards Or Libraries.

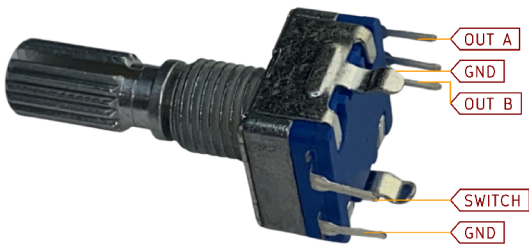


Rotary Encoder Pins

Here is the pin-out for most encoder types:

Pin	Common Label	Description
OUT A	CLK	connects to switch A inside the encoder

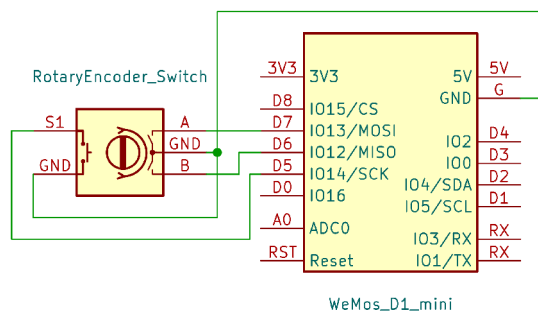
Pin	Common Label	Description
OUT B	DT	connects to switch B inside the encoder
GND		connects to the other end of both switches A and B
SWITCH	SW	connects to the (optional) third switch that is enabled by pressing the encoder knob
GND		connects to the other end of the press switch



Both **GND** pins work independently and are not connected. You must connect *both* **GND** pins to **GND**.

Obviously you can connect both **GND** pins right at the **Rotary Encoder** end and save one wire when connecting the **Rotary Encoder** to your microcontroller.

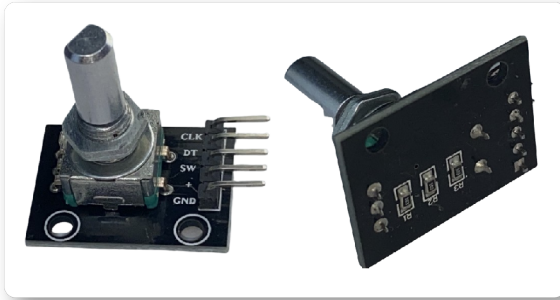
When hooking up such a mechanical **Rotary Encoder** to your microprocessor, the schematics could look like this:



A total of **four** wires and **three** GPIO pins are required, one for each physical switch.

Using Breakout Boards

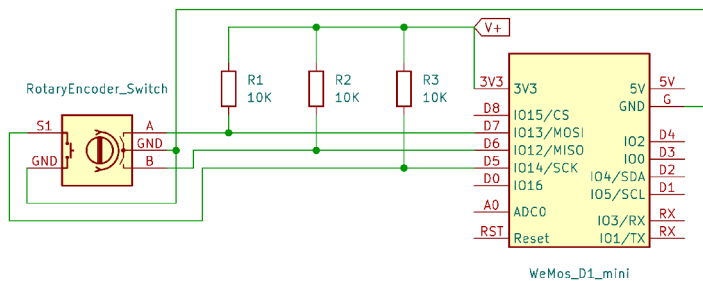
A bit easier to work with are simple and readily available breakout boards that have a mechanical **Rotary Encoder** soldered right on to them:



Breakout boards combine both physical **GND** pins to just *one* **GND** output pin. They typically add one *new* pin labelled **+** or **V+**, though, which frequently raises confusion.

You do not need to connect pin **V+** and can ignore it.

If you do connect **V+** with the positive voltage supply, this is what happens:



When positive voltage is (optionally) supplied to pin **V+**, it powers three *pullup resistors* that in turn *pull up* the output pins.

Connect **V+** only *if you need* to actively *pull up* the GPIO pins.

Microcontroller GPIOs typically have *built-in pullup resistors* that you activate by software. Then the **V+** pin is not helping in any way. Save one wire by ignoring the pin.

Live Working Example

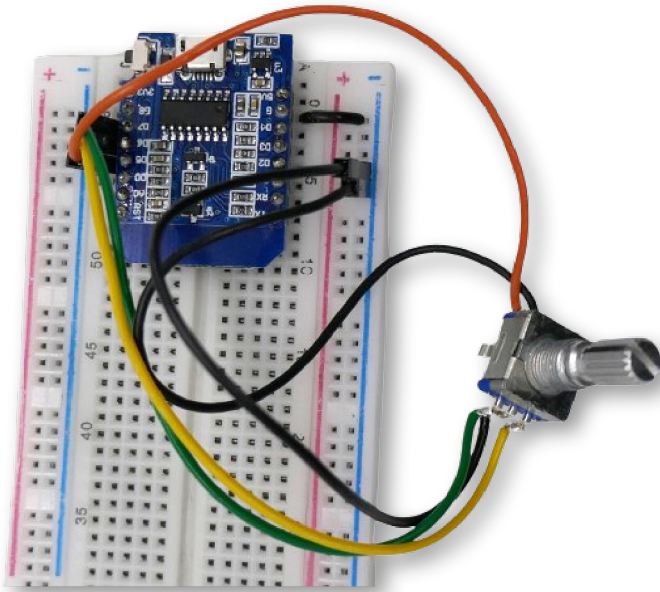
For this test, you need this:

- **Microprocessor:** The example uses a *ESP8266*.
- **Rotary Encoder:** The example uses a vanilla encoder with built-in switch (5-pin model) without a breakout board.

Wire everything according to the circuit layout depicted above. It does not matter whether you are using a pure mechanical **Rotary Encoder** or a breakout board. If you do use a breakout board, leave the **+/V+** pin unconnected.

- Connect the microcontroller **GND** (G) to both **GND** pins at the **Rotary Encoder**

- Connect the remaining three pins coming from the **Rotary Encoder** to GPIO pins of your choice. The code below assumes that you connect **Switch A/CLK** to **D7**, **Switch B/DT** to **D6**, and **Switch/SW** to **D5**.



Here is the code:



```
#include <Arduino.h>

// define input pins
// if you use a Rotary Encoder breakout board, leave pin V+ unconnected
#define CLK_PIN D7 // pin D7 connected to rotary encoder CLK (OUT A)
#define DT_PIN D6 // pin D6 connected to rotary encoder DT (OUT B)
#define SW_PIN D5 // pin D5 connected to rotary encoder SW (SWITCH)

// define modes of movement
#define DIRECTION_R 0 // right turn
#define DIRECTION_L 1 // left turn

// define state variables
int counter = 0;
int direction = DIRECTION_R;
int CLKstate;
int buttonState;
int prev_CLKstate;
int prev_buttonState;

void setup() {
  // make sure the baud rate matches your settings
  Serial.begin(115200);

  // configure pins and activate pullup
  pinMode(CLK_PIN, INPUT_PULLUP);
  pinMode(DT_PIN, INPUT_PULLUP);
  pinMode(SW_PIN, INPUT_PULLUP);

  // initialize the previous CLK state (status of internal switch A)
  prev_CLKstate = digitalRead(CLK_PIN);
  prev_buttonState = digitalRead(SW_PIN);
}

void loop() {
  // manually monitoring and post-processing rotary encoder signals

  // monitor the rotating knob:

  // check current state of switch A
  CLKstate = digitalRead(CLK_PIN);

  // any change from before? Then is it now ON (HIGH)?
  if (CLKstate != prev_CLKstate && CLKstate == HIGH) {
    // let's check switch B to find out the direction
    // in clockwise direction, A fires before B, so switch B should be HIGH
    // (HIGH = switch is NOT ON (all inputs are pulled up and connect to GND when swi
```

```
    if (digitalRead(DT_PIN) == HIGH) {
        counter++;
        direction = DIRECTION_R;
    } else {
        counter--;
        direction = DIRECTION_L;
    }

    Serial.print("Direction: ");
    if (direction == DIRECTION_R)
        Serial.print("to the right");
    else
        Serial.print("to the left");

    Serial.print(" - Increments: ");
    Serial.println(counter);
}

// remember current state for next time
prev_CLKstate = CLKstate;

// monitor the button press (pressing on knob):

// pins are HIGH by default (pulled up)
// they switch to LOW when a switch activates (connects to GND):
buttonState = digitalRead(SW_PIN);
if (buttonState != prev_buttonState)
{
    if (buttonState == LOW)
        Serial.println("The button was PRESSED.");
    else
        Serial.println("The button was RELEASED.");
    // remember current state for next time:
    prev_buttonState = buttonState;
}
}
```

The sketch defines all three GPIO pins as *INPUT_PULLUP*: the internal built-in pullup resistor is used. There is no need to externally pull up the pins.

Should you use a *rotary encoder breakout board*, leave the pin **V+** unconnected to disable the *external pullup resistors* on the breakout board.

When you upload and run the sketch, the serial output reports every movement and key press of the **Rotary Encoder**.

Slow Website?

This website is *very fast*, and pages should appear *instantly*. If this site is **slow** for you, then *your routing* may be messed up, and this issue does not only affect *done.land*, but potentially a few other websites and downloads as well. Here are simple steps **to speed up your Internet experience and fix issues with slow websites and downloads.**

Comments

Please *do leave comments* below. I am using *utteran.ce*, an *open-source* and *ad-free* light-weight commenting system.

► Here is how your comments are stored

 [Show on Github](#)  [Submit Issue](#)

(content created Feb 27, 2024)